

Dynamic Arrays and Amortized Analysis — Quiz 03

Kevin Javier Gonzalez

Iván Felipe Maluche Suarez

Angelica Pascagaza Vega

Estructuras de Datos

Grupo 3

Semestre 2026-2

Notación y modelo de costo

P1 — Arreglo dinámico desde cero

a) **Implementación.** En el archivo de `DynamicArray.java` se encuentra la implementación del arreglo dinámico.

b) **Invariantes.**

I1. $0 \leq \text{size} \leq \text{capacity}$

I2. $\text{capacity} \geq 1$

I3. $\text{index} \in [0, \text{size})$ y $\text{data}[n] == \text{null}$ para $n \in [\text{size}, \text{capacity})$

c) **Complejidad worst-case.**

Operación	Worst-case	Justificación
<code>get</code>	$\Theta(1)$	El método solamente comprueba si el <code>index</code> es válido y devuelve el dato, debido a que el array por defecto ya lo hace sin necesidad de recorrer nada cuesta $O(1)$
<code>set</code>	$\Theta(1)$	Similar a <code>get</code> , es una búsqueda de costo 1 más en este caso se asigna al valor un dato manteniendo constante la cota
<code>append</code>	$O(n)$	Debido al <code>resize</code> en el peor caso es necesario construir un nuevo array y dato por dato del anterior asignarlo en el mismo.
<code>removeLast</code>	$\Theta(1)$	Tal como el caso de <code>get</code> y <code>set</code> consiste en guardar el último valor, asignarle un nulo y disminuir <code>size</code> , siendo operaciones que me dan una cota constante.

d) **Worst-case vs. amortizado.**

Al realizar el `append` siempre que $\text{size} < \text{capacity}$ el costo de operación será "barata", lo cual permite que cuando pase el caso en que $(\text{size} = \text{capacity})$ el costo de `resize` se amortigüe en el total.

P2 — Snake sobre DynamicArray con desplazamiento

a) **Costo de `removeTail`.**

Al llamar `removeTail` en esa implementación es necesario eliminar la cola y luego reasignar los valores que quedan una posición hacia atrás, esto implica un costo $O(n)$ (con n siendo en este caso el `size`) al tener que recorrer el array.

b) **Costo total de m movimientos.**

Al ser de longitud k habría un costo de $O(k)$ lo cual al realizarse m veces nos daría un costo total de $O(km)$.

c) **Validez de la afirmación.** Esto es falso, `move()` no es amortizado porque en cada operación de `move()` tengo que hacer n (`size`) actualizaciones de posición de la serpiente lo que hace que en promedio sea $\frac{O(n \cdot n)}{n} \rightarrow O(n)$ veces el costo amortizado, a diferencia del caso de `append` donde gracias a la distribución entre operaciones caras y baratas el costo se amortigua.

d) **Bottleneck.**

El bottleneck en esta implementación consistiría en que debido a que en cada movimiento que la serpiente no consume una manzana (lo cual pasará frecuentemente) será necesario ejecutar un `RemoveTail`, el cual como se explicó en el punto anterior es $O(n)$. Esto culminaría en que esta operación básica y siempre necesaria para todo el loop jugable tenga un costo considerable y pueda generar problemas de estabilidad al ejecutarse en casi todo momento.

P3 — Rediseño: representación interna

a) Variables de estado

- **data**: arreglo nativo `Object[]` de tamaño `capacity` donde se almacenan las posiciones.
- **tail**: índice físico donde está la **cola** (índice lógico 0).
- **size**: número de elementos vivos (longitud lógica de la serpiente).
- **capacity**: tamaño físico del arreglo **data**.

La cabeza está en $(\text{tail} + \text{size} - 1) \bmod \text{capacity}$. Solo se necesitan estas cuatro variables (constante).

b) Mapeo lógico

$$\text{phys}(i) = (\text{tail} + i) \bmod \text{capacity}, \quad 0 \leq i < \text{size}.$$

Propiedades:

1. **Rango**: como $\text{tail} \in [0, C)$ y $i \in [0, \text{size}) \subset [0, C)$, se cumple $0 \leq \text{phys}(i) < C$.
2. **Inyectividad**: si $\text{phys}(i) = \text{phys}(j)$ con $0 \leq i, j < \text{size} \leq C$, entonces $i \equiv j \pmod{C}$, y como $|i - j| < C$, se tiene $i = j$.
3. **Celda libre**: si $\text{size} < C$, $\text{phys}(\text{size}) = (\text{tail} + \text{size}) \bmod C$ no coincide con ningún $\text{phys}(i)$ con $i < \text{size}$, así que esa celda está libre para el próximo `addHead`.

c) Operaciones

Costos: `addHead` $O(1)$ amortizado ($O(\text{size})$ solo cuando ejecuta `resize`); `removeTail` $O(1)$ worst-case; `get` $O(1)$.

d) `resize()`

Al copiar en el orden lógico $i = 0, 1, \dots, \text{size} - 1$ al inicio del nuevo arreglo, se preserva exactamente el mismo orden lógico, sin importar cómo estaban distribuidos físicamente antes. Tras el `resize`, `tail` = 0.

e) Invariantes

- S1**. $0 \leq \text{size} \leq \text{capacity}$ y $\text{capacity} \geq 1$.
- S2**. $0 \leq \text{tail} < \text{capacity}$.
- S3. Mapeo consistente**: `data[phys(i)]` es la posición lógica i para todo $0 \leq i < \text{size}$; en particular, `data[tail]` es la cola y `data[phys(size - 1)]` es la cabeza.
- S4. Celdas libres**: las $\text{capacity} - \text{size}$ celdas no ocupadas por `phys(0..size - 1)` son `null`.

Preservación:

- **addHead**: escribe en `phys(size)` (celda libre, S4) y aumenta `size` en 1; S1–S3 se mantienen.
- **removeTail**: lee `data[tail]`, lo pone a `null` y avanza `tail`; S1–S4 se mantienen.
- **resize**: reubica los elementos en orden lógico con `tail` = 0; S1–S4 se mantienen.

f) Traza

Estado inicial: `size` = `capacity` = 2, cola A , cabeza B , `tail` = 0, `data` = $[A, B]$.

Mov.	Acción	data	tail	size	cap.	¿Resize?
—	inicial	$[A, B]$	0	2	2	—
N_1	no come	$[\text{null}, B, C, \text{null}]$	1	2	4	Sí
N_2	come	$[\text{null}, B, C, D]$	1	3	4	No
N_3	no come	$[E, \text{null}, C, D]$	2	3	4	No
N_4	no come	$[E, F, \text{null}, D]$	3	3	4	No
N_5	come	$[E, F, G, D]$	3	4	4	No
N_6	no come	$[\text{null}, E, F, G, H, \text{null}, \text{null}, \text{null}]$	1	4	8	Sí

P4 — Análisis amortizado

a) Aggregate Method

La longitud inicial es 2 y cada manzana suma 1, pero como **addHead** va antes de **removeTail**, el tamaño pasa transitoriamente de k a $k + 1$. Por eso la longitud máxima transitoria es $n_{\text{máx}} = g + 3 = O(g)$. Con $C_0 = 2$ al ir duplicando la **capacity**, las capacidades son $2, 4, 8, \dots, 2^{r+1}$. Un **resize** se ejecuta cuando **size** = **capacity**, y en ese momento se copian exactamente C_j elementos. Como la estructura nunca supera $n_{\text{máx}}$, hay $r = O(\log g)$ **resizes** y el total de copias es:

$$\sum_{j=0}^{r-1} 2^{j+1} = 2^{r+1} - 2 < 2 n_{\text{máx}} = O(g).$$

Resultado: el total de copias es lineal en g , es decir $O(g)$.

b) Suma geométrica

La capacidad solo cambia en un **resize**, y entre el **resize** j y el $j + 1$ vale C_j . El **resize** $j \rightarrow j + 1$ se ejecuta cuando **size** = C_j , así que en ese instante el arreglo está lleno y se copian exactamente C_j elementos. Por el proceso de duplicación, esos valores son $2, 4, 8, \dots, 2^r$. La suma es geométrica de razón 2:

$$\sum_{j=0}^{r-1} 2^{j+1} = 2^{r+1} - 2 < 2 \cdot 2^r = 2 C_{r-1}.$$

El último término domina a todos los anteriores, así que la suma es proporcional al último término y no a la cantidad de términos. Como $C_{r-1} = O(g)$, el total de copias es $O(g)$.

c) Costo total y amortizado de addHead

El costo total real de m movimientos se descompone en $O(m)$ por los **addHead** (escritura + índices), $O(m)$ por los **removeTail** (leer, poner **null**, avanzar índice) y $O(g)$ por las copias de los **resizes**. Entonces $T(m) = O(m) + O(g) = O(m + g) = O(m)$, usando $g \leq m$. El costo amortizado de **addHead** es $T(m)/m = O(m + g)/m = O(1)$. Aunque un **addHead** individual cueste $O(n)$ en el peor caso por un **resize**, su costo amortizado es constante.

d) Papel de m y g

m y g miden cosas distintas. m controla el trabajo por operación: cada **MOVE** hace un **addHead** y, si no come, un **removeTail**, ambos $O(1)$; esta es la parte del costo que depende de cuántas operaciones se hacen. g controla el crecimiento: la longitud máxima transitoria es $g + 3$, la capacidad final es $O(g)$, hay $O(\log g)$ **resizes** y $O(g)$ copias; esta es la parte del costo que depende del tamaño alcanzado. La descomposición es $T(m) = O(m) + O(g)$. Como $g \leq m$, el total es $O(m)$, pero las dos fuentes son conceptualmente independientes: con $g = 0$ no hay **resizes**; con $g \approx m$ hay $O(\log m)$ **resizes** pero el costo sigue siendo lineal en m .

e) Método del potencial

Se elige $\Phi(\text{size}, C) = 2 \cdot \text{size} - C$. Justo después de un **resize** (con **size** = C_{old} , $C_{\text{new}} = 2C_{\text{old}}$) el potencial vale $\Phi = 0$; justo antes, con **size** = C , vale $\Phi = C$, suficiente para pagar las C copias. El costo amortizado es $\hat{c} = c + \Delta\Phi$.

Operación	$\Delta\Phi$	$\hat{c}$
addHead sin resize	2	$1 + 2 = 3$
addHead con resize	$2 - C$	$(C + 1) + (2 - C) = 3$
removeTail	-2	$1 - 2 = -1$

La suficiencia del potencial frente a **removeTail** se justifica porque esta operación **no libera capacidad**, así que no ejecuta **resizes** ni copias; solo reduce el potencial en 2, es decir, devuelve crédito sin consumirlo.

Como el potencial solo se gasta en los resizes (donde $\Delta\Phi \approx -C$) y cada resize ocurre cuando $\Phi = C$, el crédito acumulado siempre alcanza. En general, $\sum_{i=1}^t \hat{c}_i = \sum_{i=1}^t c_i + \Phi_t - \Phi_0$, con $\Phi_0 = 2$ y $\Phi_t \leq 2 \cdot \text{size}_t = O(g)$, de donde $\sum c_i = O(m)$, coherente con el método agregado. El costo amortizado por operación es $O(1)$.

P5 — The Adversarial Snake

a) El adversario no cambia el número de copias

Al no tener un adversario, un **resize** solo se ejecuta cuando **size** = **capacity**, y en ese instante se copian exactamente C_j elementos. El adversario controla cuándo ocurre cada **resize**, pero no cuántos ni de qué tamaño (la secuencia de capacidades $C_0, C_1, C_2, \dots$ depende solo de la política y de n). Por tanto, el total de copias $\sum C_j$ es el mismo sin importar el momento del crecimiento.

Para aumentar las copias, el adversario necesitaría poder ejecutar un **resize** con **size** < **capacity**, es decir, controlar también la política de capacidad o contraer la estructura para volver a expandirla. Sin contracción y con política fija, el número de copias queda determinado por n y α .

b–e) Comparación de políticas

- $C + 1$: se copian $2 + j$ en el resize j , luego $\sum_{j=0}^{n-3} (2 + j) = \Theta(n^2)$.
- $2C$: serie geométrica de razón 2, $\sum_{j=0}^{r-1} 2^{j+1} = 2^{r+1} - 2 < 2n = O(n)$, y como la última copia es $\Theta(n)$, también $\Omega(n)$. Total $\Theta(n)$.
- $\lceil 1.5C \rceil$: serie geométrica de razón 1.5; el último término domina, $\sum_{j=0}^{r-1} C_j = (C_r - C_0)/(1.5 - 1) = \Theta(n)$.

En los tres casos el costo amortizado por crecimiento es el total de copias dividido entre las n inserciones que provocaron crecimiento. El espacio sin usar tras el resize es $C_{\text{new}} - (C_{\text{old}} + 1)$, y su fracción se mide respecto a C_{new} .

Política	# Resizes	Total copias	Costo amortizado	Espacio sin usar tras resize
$C + 1$	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	0 (0%)
$2C$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(1)$	$\approx C/2$ ($\approx 50\%$)
$\lceil 1.5C \rceil$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(1)$	$\approx C/3$ ($\approx 33\%$)

f) Trade-off

$C + 1$ ahorra memoria (0 % libre) pero es lento ($\Theta(n^2)$ copias, $\Theta(n)$ amortizado). $2C$ es rápido ($\Theta(1)$ amortizado) pero desperdicia $\approx 50\%$ de la capacidad. $\lceil 1.5C \rceil$ es el punto intermedio: mismo orden asintótico que $2C$ ($\Theta(\log n)$ resizes, $\Theta(n)$ copias, $\Theta(1)$ amortizado) pero con más resizes en constante, a cambio de desperdiciar solo $\approx 33\%$.

Familia $C_{\text{new}} = \lceil \alpha C \rceil$

Para que la capacidad crezca se necesita $\alpha > 1$. La suma de capacidades es geométrica de razón α ; como $\alpha > 1$, el último término domina:

$$\sum_{j=0}^{r-1} C_j = \frac{C_r - C_0}{\alpha - 1} = \Theta\left(\frac{n}{\alpha - 1}\right), \quad r = \Theta(\log_{\alpha} n).$$

Para α constante y $\alpha > 1$, el total de copias es $\Theta(n/(\alpha - 1)) = \Theta(n)$ y el costo amortizado es $\Theta(1/(\alpha - 1)) = \Theta(1)$. Por tanto, duplicar no es necesario: cualquier $\alpha > 1$ constante funciona. Lo que no funciona es $\alpha = 1 + o(1)$ (crecimiento subgeométrico), porque entonces las copias dejan de ser lineales en n .